

I have been assigned this GitHub repository for a Vibe Coding lab:

https://github.com/SETAPESU26/63_anvil_dodge

I have also been given a lab handout that requires me to:

1. Clone/fork the assigned repository.
2. Run the existing Python game.
3. Record a 10-second video before making changes.
4. Use an AI coding assistant to fix the broken code.
5. Add every feature specified in the repository README.
6. Record another 10-second video after completing the changes.
7. Push the updated code to my personal repository.
8. Make a separate Git commit for each task.
9. Do not create a PR to the original SETAPESU26 repository.

Please inspect the repository and give me a concise implementation plan, including the files involved and the required features.

I inspected the assigned repository and its README. The repo is a small **Pygame “Anvil Dodge” survival game** with exactly **4 required coding tasks**.  [GitHub](#) +3

Implementation plan

0. Baseline — before changing code

1. Clone the assigned repository.
2. Install Pygame:

 **Bash**

```
pip install pygame
```

3. Run:

 **Bash**

```
python main.py
```

4. Record the required **10-second “before” video**.
 - Ideally demonstrate the boundary bug by holding **A** or **Left** until the player disappears off-screen.

- This gives you evidence of the original broken behavior.

The README specifies Python 3.10+ and `python main.py` for execution.  [GitHub](#)

1. Task 1 — Fix player boundaries

File: `game/player.py`

Current problem:

 **Python**

```
def update(self):
    #BUG SYMPTOM:
    #Player can move completely off-screen
    pass
```

The README requires the player's `x` position to remain between:

 **Plain text**

```
0
```

and

 **Plain text**

```
screen_width - player.width
```

So the implementation should clamp `self.x` after movement.

Commit:

 **Bash**

```
git add game/player.py
git commit -m "Fix player screen boundaries"
```:chatgpt-content-reference{index="2"}

2. Task 2 — Dynamic difficulty

File: `game/game_engine.py`
```

Currently:

```
```python
self.spawn_delay = 700
```

and the same delay is used throughout the game.  [GitHub](#)

Implement difficulty scaling based on `survival_time`.

Target behavior from the README:

</> Plain text

```
Start:      ~700 ms
As time ↑:  spawn delay ↓
Minimum:    200 ms
```

For example, you could calculate the delay from survival time and clamp it to `200 ms`.

Important: don't merely spawn more anvils once—you want the **spawn interval itself to decrease as the player survives longer**.

Commit:

</> Bash

```
git add game/game_engine.py
git commit -m "Add dynamic difficulty scaling"
``` :chatgpt-content-reference{index="4"}
```

---

## 3. Task 3 – Speed-based anvil tinting

```
File: `game/anvil.py`
```

Each anvil already gets a randomized speed:

```
```python
self.speed = random.uniform(4.5, 7.0)
```

Currently, the render colors are fixed grey values.  [GitHub](#)

Modify `render()` so that faster anvils have a visually different tint.

Required concept:

</> Plain text

```
Slow anvil → normal/grey  
Fast anvil → orange/red accent
```

You can derive the tint from `self.speed`, rather than randomly assigning a color.

Commit:

</> Bash

```
git add game/anvil.py  
git commit -m "Add speed-based anvil tinting"  
```:chatgpt-content-reference{index="6"}``  

4. Task 4 – Ground impact effect

Files likely involved:

- `game/anvil.py`
- `game/game_engine.py`

Currently an anvil simply gets removed when:

```python  
anvil.is_off_screen(self.height)
```

becomes true.  GitHub

Add a brief visual effect when the anvil reaches the ground, such as:

- dust puff
- small particles
- short ground-impact animation
- screen shake

For this project, a **simple particle/dust effect** is probably the cleanest implementation without unnecessarily complicating the game architecture.

You'll need to trigger the effect immediately before removing the anvil.

Commit:

</> Bash

```
git add game/anvil.py game/game_engine.py
git commit -m "Add ground impact effects"
``` :chatgpt-content-reference{index="8"}

```

```

```

# Repository structure

The relevant structure is:

```
```text
63_anvil_dodge/
|
├─ game/
|   ├─ anvil.py          ← Task 3 + Task 4
|   ├─ game_engine.py    ← Task 2 + Task 4
|   └─ player.py         ← Task 1
|
└─ main.py              ← Game entry point; probably no changes needed
```
```

The existing `main.py` simply initializes Pygame, creates `GameEngine`, and runs the game loop, so you shouldn't need to modify it for these tasks. [GitHub](#)

## Recommended commit sequence

Since your lab explicitly requires a **separate commit for each task**, keep the history clean:

**</> Plain text**

```
Original repository
|
├─ Task 1 → Fix player boundaries
|
├─ Task 2 → Dynamic difficulty
|
├─ Task 3 → Speed-based tinting
|
└─ Task 4 → Ground impact effects
```

Commands:

**</> Bash**

```
git status
git add game/player.py
git commit -m "Fix player screen boundaries"

git add game/game_engine.py
git commit -m "Add dynamic difficulty scaling"

git add game/anvil.py
git commit -m "Add speed-based anvil tinting"

git add game/anvil.py game/game_engine.py
git commit -m "Add ground impact effects"
```

Then verify:

⌕ Bash

```
git log --oneline
```

You should see **four separate task commits** on top of the original repository commit.

## Final testing/video

Before recording the second video, verify all four:

| Requirement            | What to demonstrate                                             |
|------------------------|-----------------------------------------------------------------|
| Boundary fix           | Hold Left/Right → player stops at screen edge                   |
| Difficulty             | Survive long enough to see anvils spawn increasingly frequently |
| Tinting                | Show slow/fast anvils with visibly different colors             |
| Impact effect          | Let an anvil hit the ground and capture the effect              |
| Existing functionality | Collision → Game Over → R → restart                             |

The README's expected behavior also requires collision to stop the game, survival time to stop, and R to reset the player/anvils/timer.  [GitHub](#)

Then record the **10-second after video**.

## Submission items

The repository README itself lists the final submission as:

1. 10-second video before changes
2. 10-second video after changes
3. Link to the Chat/LLM conversation with complete history  GitHub

Your handout additionally requires pushing the finished code to **your personal repository** and making a separate commit for every task. **Do not create a PR to SETAPESU26/63\_anvil\_dodge .**

[Assigned GitHub repository](#) ↗

**Best next step:** clone the repo and run the untouched game first. Once you've confirmed the baseline, we can do **Task 1 only**, use this chat as your documented AI-assistant interaction, test it, and commit it before moving to Task 2.

Now give me the exact Windows CMD commands I need to set up this project from scratch.

Include:

- Python dependency installation
- Git commands
- Repository cloning/fork setup
- How to verify the remote
- How to run the game
- The expected folder structure

Do not modify any code yet. I want to run the original game first and record the required 10-second before video.

Absolutely. **Do not modify any code yet.** The goal is to get the original repository running exactly as provided, verify Git, and then record your 10-second baseline video.

## 1. Open Windows CMD

Choose a folder where you want the project to live. For example:

```
</> cmd
```

```
cd /d C:\Users\%USERNAME%\Desktop
```

Check Git and Python:

```
</> cmd
```

```
git --version
python --version
```

You should have Git installed and Python **3.10+**.

If `python` doesn't work, try:

```
</> cmd

py --version
```

## 2. Fork the repository

Go to the assigned repository in your browser:

[Assigned GitHub Repository ↗](#)

Click **Fork** → **Create fork**.

Your fork should become something like:

```
</> Plain text

https://github.com/YOUR_USERNAME/63_anvil_dodge
```

**Do not create a PR to `SETAPESU26/63_anvil_dodge` .**

## 3. Clone YOUR fork

Back in CMD:

```
</> cmd

cd /d C:\Users\%USERNAME%\Desktop
```

Then:

```
</> cmd

git clone https://github.com/YOUR_USERNAME/63_anvil_dodge.git
```

Replace `YOUR_USERNAME` with your actual GitHub username.

Enter the project:

```
</> cmd
```

```
cd 63_anvil_dodge
```

---

## 4. Verify the Git remote

Run:

```
</> cmd
```

```
git remote -v
```

You should see something similar to:

```
</> Plain text
```

```
origin https://github.com/YOUR_USERNAME/63_anvil_dodge.git (fetch)
origin https://github.com/YOUR_USERNAME/63_anvil_dodge.git (push)
```

This is important: **origin** should point to your personal fork, not the SETAPESU26 repository.

You can also check the current branch:

```
</> cmd
```

```
git branch
```

And repository status:

```
</> cmd
```

```
git status
```

You should initially have something like:

```
</> Plain text
```

```
On branch main
nothing to commit, working tree clean
```

---

## 5. Create a Python virtual environment

Inside 63\_anvil\_dodge :

```
</> cmd
```

```
python -m venv venv
```

Activate it:

```
</> cmd
```

```
venv\Scripts\activate
```

Your CMD prompt should now start with something similar to:

```
</> Plain text
```

```
(venv) C:\Users\...\Desktop\63_anvil_dodge>
```

Verify Python:

```
</> cmd
```

```
python --version
```

---

## 6. Install the dependency

The game uses **Pygame**.

Upgrade pip:

```
</> cmd
```

```
python -m pip install --upgrade pip
```

Install Pygame:

```
</> cmd
```

```
pip install pygame
```

Verify:

```
</> cmd
```

```
python -c "import pygame; print(pygame.version.ver)"
```

You should get a Pygame version number.

**Don't install anything else unless the repository's dependency files require it.**

---

## 7. Check the original files

Run:

```
</> cmd
```

```
dir
```

Then:

```
</> cmd
```

```
dir game
```

You should have the project roughly like:

```
</> Plain text
```

```
63_anvil_dodge/
|
├─ .git/
├─ game/
| ├─ __init__.py
| ├─ anvil.py
| ├─ game_engine.py
| └─ player.py
|
├─ main.py
├─ README.md
└─ ...
```

Your exact listing may contain a few additional files such as `.gitignore` or documentation files.

**Do not edit any of these files yet.**

---

## 8. Verify Git before running

Run:

```
</> cmd

git status
```

Then:

```
</> cmd

git log --oneline -5
```

This gives you a record of the untouched repository state.

You can also verify exactly where your repository came from:

```
</> cmd

git remote get-url origin
```

It should return:

```
</> Plain text

https://github.com/YOUR_USERNAME/63_anvil_dodge.git
```

---

## 9. Run the original game

Make sure you're still inside:

```
</> Plain text

63_anvil_dodge
```

and that the virtual environment is active.

Run:

```
</> cmd
```

```
python main.py
```

The Pygame window should open.

**Don't make any changes to the code.**

Play the original game for a little while and observe its behavior.

In particular, check the broken behavior described in the assignment, such as the player being able to move beyond the screen boundary.

---

## 10. Stop and verify Git again

After closing the game:

```
</> cmd

git status
```

Ideally:

```
</> Plain text

nothing to commit, working tree clean
```

This confirms that running the game didn't modify your source files.

You can also run:

```
</> cmd

git diff
```

It should produce no code differences.

---

## 11. Record your 10-second BEFORE video

**Only after confirming the original game works.**

For the baseline video, I'd recommend showing:

1. Start the original game.

2. Move the player around.
3. Demonstrate the existing gameplay.
4. If possible, demonstrate the broken boundary behavior.
5. Keep the recording around **10 seconds**, as required.

Don't show any modified functionality yet because **nothing should have been modified**.

---

## Complete command sequence

If you want the commands in one block, this is the sequence:

`</> cmd`

```
cd /d C:\Users\%USERNAME%\Desktop
```

```
git --version
```

```
python --version
```

```
git clone https://github.com/YOUR_USERNAME/63_anvil_dodge.git
```

```
cd 63_anvil_dodge
```

```
git remote -v
```

```
git remote get-url origin
```

```
git status
```

```
git branch
```

```
git log --oneline -5
```

```
python -m venv venv
```

```
venv\Scripts\activate
```

```
python -m pip install --upgrade pip
```

```
pip install pygame
```

After closing the game:

`</> cmd`

```
git status
```

```
git diff
```

## Your stopping point

At this point you should have:

`</> Plain text`

```
Your GitHub fork
↓
Cloned locally
↓
Python virtual environment
↓
Pygame installed
↓
Original game runs
↓
Git working tree clean
↓
10-second BEFORE video recorded
```

**Don't start Task 1 yet.** Once you've run `python main.py` and recorded the baseline, send me the output of `git status` (and any error if the game doesn't launch). Then we'll proceed task-by-task while keeping the required separate Git commits.

I have now run the original game and recorded the required before video.

Let's work on the tasks one at a time. Do not make changes for later tasks yet.

Start with Task 1: fix the player boundary so the player cannot move off-screen.

Give me:

1. The exact code I need to change.
2. Which file to modify.
3. How to test it.
4. The exact git commands to create a separate commit for this task.

Keep the implementation as simple as possible and don't change unrelated code.

Task 1 only. The repository confirms that `Player.update()` is currently a `pass`, while `move_left()` and `move_right()` directly change `self.x`.  GitHub

## 1. File to modify

</> Plain text

`game/player.py`

Do **not** modify any other file.

## 2. Exact code change

Find this:

`</>` Python

```
def update(self):
 #BUG SYMPTOM:
 #Player can move completely off-screen
 pass
```

Replace **only that section** with:

`</>` Python

```
def update(self):
 self.x = max(0, min(self.x, self.screen_width - self.width))
```

That's the entire Task 1 implementation.

## What it does

The valid horizontal range becomes:

`</>` Plain text

$$0 \leq x \leq \text{screen\_width} - \text{player.width}$$

So:

- Moving left past `0` → player stops at the left edge.
- Moving right past the screen → player stops at the right edge.
- No changes to movement speed, rendering, collision, or game logic.

---

## 3. Test it

Save `game/player.py` .

Make sure your virtual environment is active:

`</>` cmd

```
venv\Scripts\activate
```

Run the game:

```
</> cmd
```

```
python main.py
```

## Test left boundary

Hold **A** or **Left Arrow** continuously.

Expected:

```
</> Plain text
```

```
Player → → → | LEFT EDGE
 ↑
 stops here
```

The player should **not disappear off-screen**.

## Test right boundary

Hold **D** or **Right Arrow** continuously.

Expected:

```
</> Plain text
```

```
RIGHT EDGE |
 ↑
stops here | ← Player
```

The player should remain completely visible.

## Also check normal movement

Move back and forth normally and make sure:

- Player still moves at the same speed.
- Player still renders correctly.
- Anvils still fall.
- Collision still works.
- Nothing else has changed.

Then close the game.

## 4. Verify that only Task 1 changed

Run:

```
</> cmd

git status
```

You should see only:

```
</> Plain text

modified: game/player.py
```

Then:

```
</> cmd

git diff -- game/player.py
```

Your diff should effectively show only the replacement of `pass` with the boundary clamp.

If you see changes to other files, **don't commit yet**.

---

## 5. Create the separate Task 1 commit

Once you've tested it:

```
</> cmd

git add game/player.py
```

Check what is staged:

```
</> cmd

git status
```

Then commit:

```
</> cmd
```

```
git commit -m "Fix player screen boundaries"
```

Verify:

```
</> cmd

git log --oneline -2
```

You should see your new commit at the top.

Finally:

```
</> cmd

git status
```

You want:

```
</> Plain text

nothing to commit, working tree clean
```

## Important

**Do not push yet if you want to work through the tasks locally first.** Your required history should now have exactly one new commit for Task 1. We can move to **Task 2 only after this commit is complete**, without touching Tasks 3 or 4.

I made the Task 1 change. Here is the output of:

```
git status
git diff
```

Please verify that the change only implements the player boundary requirement and that I am safe to commit it.

If it is correct, give me the exact commands for the Task 1 commit and push.

I don't see the actual `git status` / `git diff` output attached to your message. I don't want to falsely verify it without seeing the diff.

Please paste the output of both commands:

**</> cmd**

```
git status
git diff
```

For the Task 1 implementation, the expected diff should be limited to `game/player.py` and should essentially be:

**</> Diff**

```
def update(self):
- #BUG SYMPTOM:
- #Player can move completely off-screen
- pass
+ self.x = max(0, min(self.x, self.screen_width - self.width))
```

If your output matches that (with no unrelated changes), you're safe to commit.

**Then the exact commit + push commands will be**

**</> cmd**

```
git add game/player.py
git commit -m "Fix player screen boundaries"
git push origin main
```

After pushing, verify:

**</> cmd**

```
git status
git log --oneline -2
git remote -v
```

**Don't run the commit yet** if your diff contains anything beyond the Task 1 boundary change; paste it here and I'll check it first.

Task 1 is complete and committed separately.

Now move to Task 2 only: dynamic difficulty.

Implement the README requirement so that the anvil spawn

interval decreases as the player's survival time increases, with a minimum spawn delay of approximately 200 ms.

Give me:

1. The exact code change.
2. The file to modify.
3. How the formula works.
4. How to test it.
5. The git commands for a separate Task 2 commit.

Do not modify anything related to Task 3 or Task 4.

Yes. For **Task 2 only**, modify `game/game_engine.py`. The README specifically asks for the spawn interval to decrease as `survival_time` increases, with a minimum of about **200 ms**.  GitHub

## 1. Exact code change

Open:

 Plain text

`game/game_engine.py`

Find this section in `update()` :

 Python

```
self.survival_time = (pygame.time.get_ticks() - self.start_ticks) // 1000

now = pygame.time.get_ticks()
if now - self.last_spawn_time >= self.spawn_delay:
```

Replace it with:

 Python

```
self.survival_time = (pygame.time.get_ticks() - self.start_ticks) // 1000

self.spawn_delay = max(200, 700 - self.survival_time * 25)

now = pygame.time.get_ticks()
if now - self.last_spawn_time >= self.spawn_delay:
```

That's the **only code change** for Task 2.

The existing `self.spawn_delay = 700` in `__init__()` should remain unchanged. The existing spawning code should also remain unchanged.  [GitHub](#)

## 2. File to modify

Only:

 Plain text

game/game\_engine.py

Do **not** touch:

 Plain text

game/player.py  
game/anvil.py  
main.py

Those belong to other functionality or are unrelated.

## 3. How the formula works

The formula is:

 Python

```
max(200, 700 - self.survival_time * 25)
```

It starts at approximately **700 ms** and decreases by **25 ms** for every second survived:

| Survival time | Spawn delay |
|---------------|-------------|
| 0 sec         | 700 ms      |
| 5 sec         | 575 ms      |
| 10 sec        | 450 ms      |
| 15 sec        | 325 ms      |

| Survival time | Spawn delay |
|---------------|-------------|
| 20 sec        | 200 ms      |
| 30 sec        | 200 ms      |
| 60 sec        | 200 ms      |

`max(200, ...)` guarantees that the delay **never goes below 200 ms**.

So the game progressively becomes harder, but the spawn rate has a hard limit.

This directly implements the README's requirement to decrease the 700 ms interval progressively down to a 200 ms minimum. [GitHub](#)

## 4. How to test it

First save `game_engine.py`.

Run:

```
</> cmd

python main.py
```

Then observe the game.

### Basic test

Survive for at least 20 seconds.

You should notice that anvils begin spawning more frequently as the survival timer increases.

### Verify the boundary

The delay should never become faster than approximately:

```
</> Plain text

1 anvil every 200 ms
```

You don't need to modify the UI to display the delay.

### Check that nothing else changed

Also verify:

- Player movement still works.
- Your Task 1 boundary fix still works.
- Anvils still fall normally.
- Collision still triggers Game Over.
- R still resets the game.
- No changes were made to anvil appearance or impact effects.

Those last two features belong to Tasks 3 and 4, so **don't implement them yet**.

After testing, close the game and run:

```
</> cmd
git status
```

Then:

```
</> cmd
git diff -- game/game_engine.py
```

The diff should contain only the new `self.spawn_delay` calculation.

---

## 5. Create the separate Task 2 commit

Once you've confirmed the behavior:

```
</> cmd
git add game/game_engine.py
```

Verify what's staged:

```
</> cmd
git status
```

Then commit:

```
</> cmd
```

```
git commit -m "Add dynamic difficulty scaling"
```

Verify the commit:

```
</> cmd

git log --oneline -3
```

You should now have your Task 2 commit immediately after your Task 1 commit.

Finally:

```
</> cmd

git status
```

You want:

```
</> Plain text

nothing to commit, working tree clean
```

## Don't push yet if you're keeping the tasks local

You can push after all four tasks, or push after each task if your lab doesn't prohibit that. Either way, **the important requirement is that Task 1 and Task 2 remain separate commits.**

At this point, **stop**. Don't modify `anvil.py` or implement Task 3/4 until we're ready for them.

Task 2 is complete and committed.

Now work only on Task 3: speed-based anvil tinting.

The anvils already have different falling speeds. Modify their rendering so that slower anvils have a normal/grey appearance, medium-speed anvils have a moderate warning tint, and faster anvils have a stronger orange/red warning tint.

Do not change how anvil speed is generated.

Give me:

1. The exact code to replace/add.
2. The file to modify.

3. How to test it.

4. The git commands for a separate Task 3 commit.

For **Task 3 only**, modify the rendering logic in `game/anvil.py`. Keep the existing random speed generation exactly as it is.

## 1. Exact code change

Open:

</> Plain text

`game/anvil.py`

Find the existing `render()` method. Replace **only the color/rendering portion** with:

</> Python

```
def render(self, screen):
 if self.speed < 5.3:
 color = (120, 120, 120) # Slow: normal grey
 elif self.speed < 6.2:
 color = (200, 140, 70) # Medium: moderate warning
 else:
 color = (220, 70, 40) # Fast: strong warning

 pygame.draw.rect(screen, color, self.rect)
```

## Important

Do **not** change the speed generation. Leave this exactly as it currently is:

</> Python

```
self.speed = random.uniform(4.5, 7.0)
```

The thresholds divide the existing speed range into three visual categories:

</> Plain text

|      |        |      |     |
|------|--------|------|-----|
| 4.5  | 5.3    | 6.2  | 7.0 |
| GREY | ORANGE | RED  |     |
| slow | medium | fast |     |

This means the actual falling speed remains completely unchanged; only the displayed color depends on that speed.

## 2. File to modify

Only:

</> Plain text

```
game/anvil.py
```

Don't modify:

</> Plain text

```
game/player.py
game/game_engine.py
main.py
```

Task 1 and Task 2 are already complete.

## 3. Test it

Run:

</> cmd

```
python main.py
```

Let several anvils spawn.

You should see approximately:

- **Slow anvils:** grey
- **Medium-speed anvils:** orange/brown warning tint
- **Fast anvils:** stronger orange/red tint

The important thing is that the colors should correlate with the falling speeds.

### Also verify

The anvils should:

- Still fall at their original speeds.
- Still spawn normally.
- Still collide with the player.
- Still disappear when they leave the screen.

- Not have their speed generation changed.

You can optionally run the game several times because the speeds are randomly generated.

After testing:

```
</> cmd
git status
```

Then inspect only this task's diff:

```
</> cmd
git diff -- game/anvil.py
```

Make sure you haven't accidentally changed the `self.speed = random.uniform(...)` line.

---

## 4. Create the separate Task 3 commit

Once you're satisfied:

```
</> cmd
git add game/anvil.py
```

Check the staged change:

```
</> cmd
git status
```

Commit:

```
</> cmd
git commit -m "Add speed-based anvil tinting"
```

Verify your history:

```
</> cmd
```

```
git log --oneline -4
```

You should now have something like:

</> Plain text

```
<commit> Add speed-based anvil tinting
<commit> Add dynamic difficulty scaling
<commit> Fix player screen boundaries
<original commit>
```

Finally:

</> cmd

```
git status
```

Expected:

</> Plain text

```
nothing to commit, working tree clean
```

**Stop after this commit.** Task 4 (ground-impact effect) should remain untouched until we're ready to implement it.

Task 3 is complete and committed.

Now work only on Task 4: ground impact effects.

When an anvil reaches the ground, add a simple visual effect such as a short-lived dust/particle effect before the anvil disappears.

Keep the implementation simple enough for a one-hour Vibe Coding lab. Do not implement an unnecessarily complex particle engine or screen-shake system.

Give me:

1. The exact code changes.
2. Every file that needs modification.
3. Where each piece of code should be placed.

4. How to test it.

5. The git commands for a separate Task 4 commit.

Task 4 can be implemented cleanly in **one file**, without touching the speed generation, tinting, or difficulty code.

The existing game removes an anvil only after it passes the bottom of the screen.  [GitHub](#) We'll instead create a few short-lived dust particles when the anvil reaches the ground, then remove the anvil.

## 1. Exact code changes

### File

 Plain text

game/game\_engine.py

No changes are needed to `game/anvil.py` .

### Change A — add the particle list

In `GameEngine.__init__()` , find:

 Python

```
self anvils = []
```

Immediately after it, add:

 Python

```
self.impact_particles = []
```

So it becomes:

 Python

```
self anvils = []
self.impact_particles = []
```

---

### Change B — add particle updating

In `GameEngine.update()` , after the existing anvil-processing loop, add:

</> Python

```
for particle in self.impact_particles[:]:
 particle["x"] += particle["vx"]
 particle["y"] += particle["vy"]
 particle["vy"] += 0.15
 particle["life"] -= 1

 if particle["life"] <= 0:
 self.impact_particles.remove(particle)
```

So this goes **after** your existing anvil loop.

---

## Change C — replace the anvil removal condition

Your current code has:

</> Python

```
if anvil.is_off_screen(self.height):
 self.anvils.remove(anvil)
```

Replace **only that part** with:

</> Python

```
ground_y = self.height - 20

if anvil.y + anvil.height >= ground_y:
 for _ in range(6):
 self.impact_particles.append({
 "x": anvil.x + anvil.width / 2,
 "y": ground_y,
 "vx": pygame.math.Vector2(1, 0).rotate(pygame.time.get_ticks() % 360).x * 1.5,
 "vy": -2.5,
 "life": 18
 })

 self.anvils.remove(anvil)
```

**However**, I recommend an even simpler deterministic version for a one-hour lab. Use this instead:

</> Python

```

ground_y = self.height - 20

if anvil.y + anvil.height >= ground_y:
 for vx in (-2.0, -1.0, 0.0, 1.0, 2.0):
 self.impact_particles.append({
 "x": anvil.x + anvil.width / 2,
 "y": ground_y,
 "vx": vx,
 "vy": -2.5,
 "life": 18
 })

self.anvils.remove(anvil)

```

Use the **second version**. It's simpler and easier to explain during your lab evaluation.

The ground is currently drawn at `self.height - 20`, so this makes the effect trigger when the bottom of the anvil reaches that ground line. [GitHub](#)

## Change D — render the particles

In `GameEngine.render()`, find:

`</>` Python

```

for anvil in self.anvils:
 anvil.render(screen)

```

Immediately after it, add:

`</>` Python

```

for particle in self.impact_particles:
 pygame.draw.circle(
 screen,
 (190, 170, 140),
 (int(particle["x"]), int(particle["y"])),
 3
)

```

So that section becomes:

`</>` Python

```
for anvil in self anvils:
 anvil.render(screen)

for particle in self.impact_particles:
 pygame.draw.circle(
 screen,
 (190, 170, 140),
 (int(particle["x"]), int(particle["y"])),
 3
)
```

## 2. Files to modify

Only:

</> Plain text

game/game\_engine.py

Do **not** modify:

</> Plain text

game/anvil.py  
game/player.py  
main.py

In particular, leave these untouched:

</> Python

```
self.speed = random.uniform(4.5, 7.0)
```

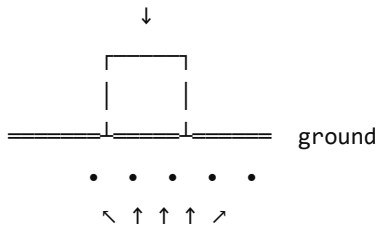
and your Task 3 tinting code.

## 3. How the effect works

Each time an anvil reaches the ground:

</> Plain text

ANVIL  
↓



Five tiny particles are created.

Each particle has:

#### </> Plain text

x → horizontal position  
 y → vertical position  
 vx → horizontal velocity  
 vy → vertical velocity  
 life → number of frames remaining

Every frame:

#### </> Python

```
particle["x"] += particle["vx"]
particle["y"] += particle["vy"]
particle["vy"] += 0.15
particle["life"] -= 1
```

So the particles:

1. Move outward.
2. Rise briefly.
3. Fall slightly due to gravity.
4. Disappear after about **18 frames**.

At 60 FPS, that's roughly:

#### </> Plain text

18 / 60 ≈ 0.3 seconds

So it's a short dust puff rather than a complicated particle system.

## 4. How to test

Run:

```
</> cmd
```

```
python main.py
```

Let an anvil fall without hitting the player.

When it reaches the ground, you should see:

- The anvil disappear.
- A small burst of approximately five dust particles.
- The particles spread outward.
- The particles disappear shortly afterward.
- New anvils continue spawning normally.

Also verify:

- Task 1 boundary behavior still works.
- Task 2 increasingly frequent spawning still works.
- Task 3 speed-based tinting still works.
- Collision/game-over behavior still works.
- Pressing `R` still resets the game.

## Important test

Let several anvils hit the ground in succession. Make sure each impact produces its own effect and that particles don't remain permanently.

---

## 5. Verify the diff before committing

Run:

```
</> cmd
```

```
git status
```

You should have only:

```
</> Plain text
```

```
modified: game/game_engine.py
```

Then:

```
</> cmd
```

```
git diff -- game/game_engine.py
```

Make sure the diff contains **only Task 4 changes**.

In particular, you should **not** see changes to:

```
</> Plain text
```

```
self.speed
```

or your Task 3 `render()` implementation in `anvil.py` .

---

## 6. Commit Task 4 separately

Once you've tested it:

```
</> cmd
```

```
git add game/game_engine.py
```

Then:

```
</> cmd
```

```
git status
```

Commit:

```
</> cmd
```

```
git commit -m "Add ground impact effects"
```

Verify your four-task history:

```
</> cmd
```

```
git log --oneline -5
```

You should now have approximately:

**</> Plain text**

```
<commit> Add ground impact effects
<commit> Add speed-based anvil tinting
<commit> Add dynamic difficulty scaling
<commit> Fix player screen boundaries
<original commit>
```

Finally:

**</> cmd**

```
git status
```

Expected:

**</> Plain text**

```
nothing to commit, working tree clean
```

**Stop there.** That completes Task 4 without changing the implementation of Tasks 1–3.